Machine Boss: Dynamic Programming Performance Benchmarks

Machine Boss benchmark suite

March 4, 2026

1 Methodology

We benchmark the Forward and Viterbi dynamic programming algorithms across multiple backends in the Machine Boss WFST toolkit:

- **C++ native**: the `boss` CLI (`-L` for Forward, `-V` for Viterbi)
- **JAX 1D scan**: Python/JAX using `jax.lax.scan` (sequential)
- **JAX 1D parallel**: Python/JAX using `jax.lax.associative_scan` (parallel prefix)
- **JAX 2D scan**: Python/JAX 2D with sequential outer scan
- **JAX 2D wavefront**: Python/JAX 2D with anti-diagonal wavefront parallelism
- **JAX GPU**: JAX on GPU (skipped if unavailable)
- **JS CPU**: JavaScript CPU fallback via Node.js

1.1 1D problems (generators)

Random generator machines (output-only) with DNA alphabet (A, C, G, T) are tested across:

- Sequence length $L \in \{100, 500, 2000, 10000\}$
- Machine states $S \in \{4, 16, 64\}$

1.2 2D problems (transducers)

Random transducer machines with DNA alphabet are tested across:

- Input length $L_{\text{in}} \in \{10, 50, 200\}$
- Output length $L_{\text{out}} \in \{10, 50, 200\}$
- Machine states $S \in \{4, 16, 64\}$

Each configuration is timed over multiple repetitions after a warmup call (which also triggers JIT compilation for JAX backends). We report mean $\pm$ standard deviation in seconds. Configurations that exceed a timeout threshold are recorded with their probe timing and larger sizes are skipped.

2 Hardware

Property	Value
wideboy.local / Intel(R) Xeon(R) W-3223 CPU @ 3.50GHz	
System	Darwin wideboy.local 24.1.0 Darwin Kernel Version 24.1.0: Thu Oct 10 21:02:27 PDT 2024; root:xnu-11215.41.3~2/RELEASE_X86_64 x86_64
CPU	Intel(R) Xeon(R) W-3223 CPU @ 3.50GHz
Physical cores	8
Logical cores	16
CPU frequency	3.50 GHz
Memory	32 GB
Python	3.12.6
Platform	macOS-15.1.1-x86_64-i386-64bit

3 Results: 1D Forward

Table 1: 1D Forward timings (generators) (seconds, mean ± std).

S	L	`cpp`	`jax_1d_optimal`	`jax_1d_simple`	`js_cpu`
4	100	0.038 ± 0.001	0.109 ± 0.010	0.172 ± 0.018	0.0018 ± 0.0004
4	500	0.062 ± 0.001	0.111 ± 0.012	0.206 ± 0.003	0.0044 ± 0.0031
4	2000	0.088 ± 0.001	0.113 ± 0.008	0.234 ± 0.040	0.010 ± 0.006
4	10000	0.124 ± 0.002	0.154 ± 0.018	0.217 ± 0.029	0.036 ± 0.006
16	100	0.142 ± 0.001	0.126 ± 0.025	0.228 ± 0.030	0.0065 ± 0.0058
16	500	0.168 ± 0.001	0.153 ± 0.010	0.252 ± 0.013	0.016 ± 0.007
16	2000	0.190 ± 0.002	0.257 ± 0.007	0.252 ± 0.026	0.052 ± 0.007
16	10000	0.225 ± 0.003	0.896 ± 0.029	0.372 ± 0.014	0.254 ± 0.015
64	100	0.236 ± 0.002	0.506 ± 0.016	0.280 ± 0.028	0.040 ± 0.007
64	500	0.252 ± 0.006	2.72 ± 0.29	0.319 ± 0.018	0.162 ± 0.009
64	2000	0.240 ± 0.006	14.77 ± 0.45	0.608 ± 0.042	0.622 ± 0.035
64	10000	0.269 ± 0.002	189.90 ± 0.00	2.81 ± 0.03	2.89 ± 0.01

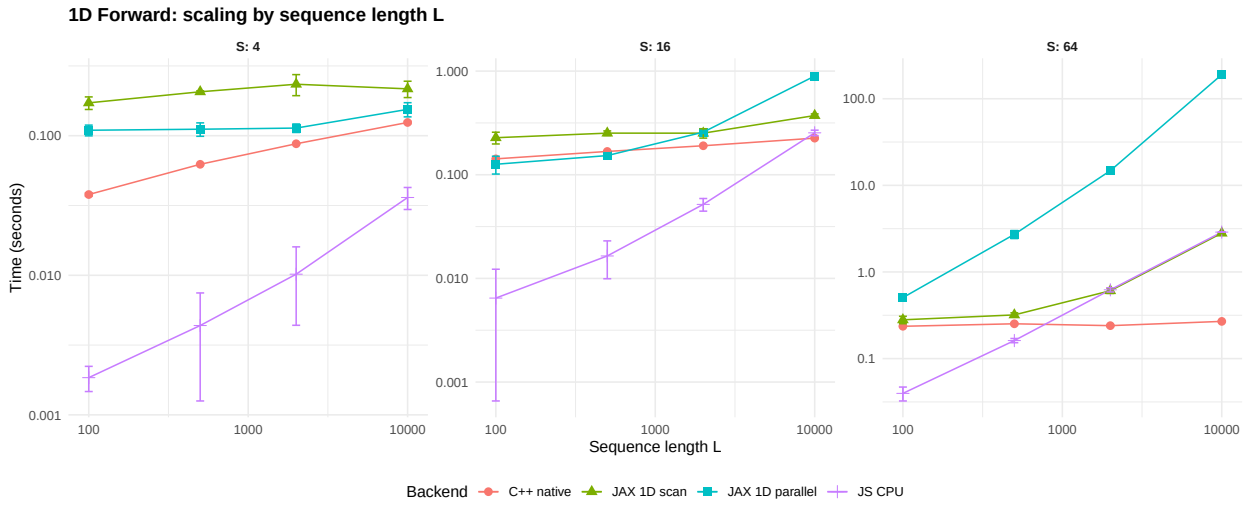

Figure 1: 1D Forward: scaling by sequence length L, faceted by states S. Log-log scale.

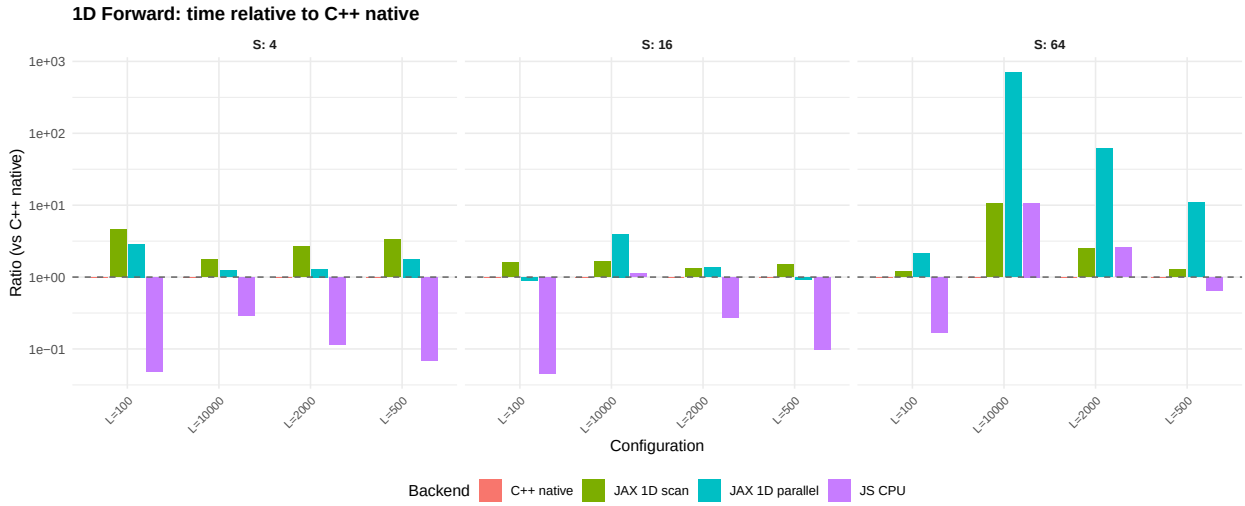

Figure 2: 1D Forward: time relative to C++ native, by configuration.

4 Results: 1D Viterbi

Table 2: 1D Viterbi timings (generators) (seconds, mean ± std).

S	L	cpp	jax_1d_optimal	jax_1d_simple	js_cpu
4	100	0.061 ± 0.000	0.054 ± 0.002	0.112 ± 0.004	0.0016 ± 0.0002
4	500	0.087 ± 0.001	0.060 ± 0.006	0.104 ± 0.008	0.0042 ± 0.0025
4	2000	0.120 ± 0.001	0.062 ± 0.003	0.099 ± 0.003	0.0097 ± 0.0040
4	10000	0.172 ± 0.001	0.072 ± 0.003	0.138 ± 0.005	0.031 ± 0.007
16	100	0.161 ± 0.002	0.059 ± 0.002	0.126 ± 0.028	0.0065 ± 0.0034
16	500	0.188 ± 0.001	0.059 ± 0.001	0.117 ± 0.006	0.018 ± 0.008
16	2000	0.219 ± 0.007	0.116 ± 0.003	0.130 ± 0.015	0.053 ± 0.006
16	10000	0.266 ± 0.001	0.365 ± 0.006	0.169 ± 0.007	0.246 ± 0.006
64	100	0.255 ± 0.006	0.219 ± 0.005	0.156 ± 0.021	0.040 ± 0.010
64	500	0.222 ± 0.004	0.931 ± 0.005	0.175 ± 0.023	0.161 ± 0.007
64	2000	0.216 ± 0.002	5.08 ± 0.10	0.250 ± 0.006	0.592 ± 0.018
64	10000	0.175 ± 0.001	26.79 ± 0.28	1.12 ± 0.01	2.89 ± 0.06

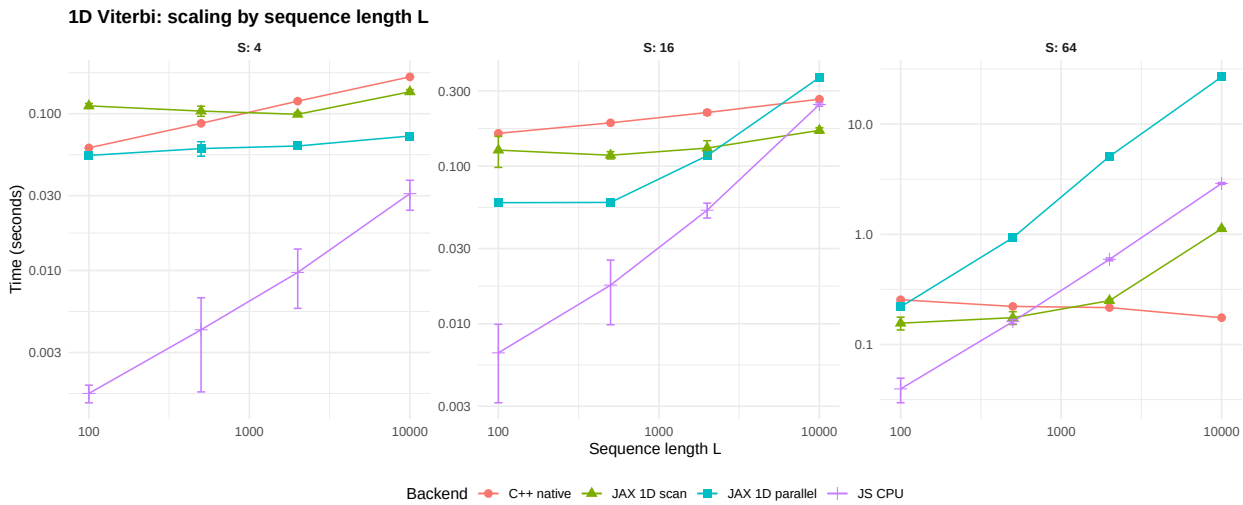

Figure 3: 1D Viterbi: scaling by sequence length L, faceted by states S. Log-log scale.

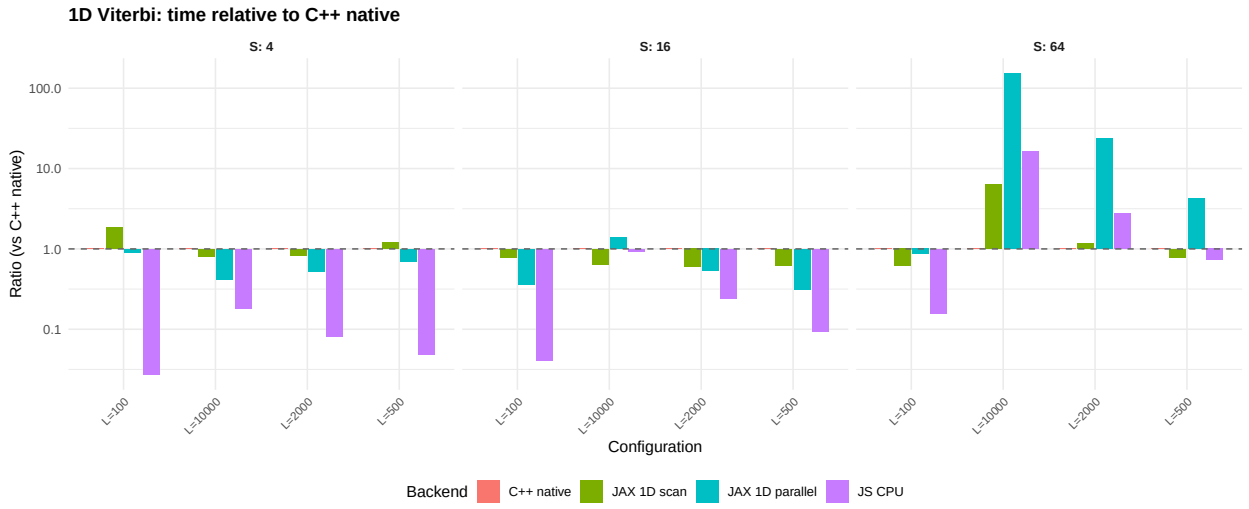

Figure 4: 1D Viterbi: time relative to C++ native, by configuration.

5 Results: 2D Forward

Table 3: 2D Forward timings (transducers) (seconds, mean ± std).

S	L_{in}	L_{out}	cpp	jax_2d_optimal	jax_2d_simple	js_cpu
4	10	10	0.085 ± 0.001	0.497 ± 0.096	0.601 ± 0.040	0.0027 ± 0.0004
4	10	50	0.105 ± 0.002	0.451 ± 0.014	1.01 ± 0.35	0.0046 ± 0.0027
4	10	200	0.130 ± 0.000	0.539 ± 0.043	0.880 ± 0.064	0.014 ± 0.003
4	50	10	0.143 ± 0.002	0.477 ± 0.016	0.788 ± 0.201	0.0048 ± 0.0029
4	50	50	0.156 ± 0.001	0.423 ± 0.031	0.823 ± 0.021	0.015 ± 0.003
4	50	200	0.177 ± 0.002	0.660 ± 0.251	0.884 ± 0.051	0.054 ± 0.005
4	200	10	0.182 ± 0.002	0.447 ± 0.009	0.675 ± 0.060	0.014 ± 0.004
4	200	50	0.215 ± 0.016	0.472 ± 0.037	0.806 ± 0.061	0.054 ± 0.005
4	200	200	0.230 ± 0.002	0.813 ± 0.324	1.21 ± 0.06	0.197 ± 0.012
16	10	10	0.228 ± 0.002	0.585 ± 0.057	0.853 ± 0.046	0.0073 ± 0.0038
16	10	50	0.245 ± 0.002	0.479 ± 0.023	0.935 ± 0.104	0.025 ± 0.005
16	10	200	0.264 ± 0.003	0.792 ± 0.301	1.21 ± 0.13	0.089 ± 0.005
16	50	10	0.278 ± 0.002	0.495 ± 0.020	0.758 ± 0.072	0.024 ± 0.005
16	50	50	0.302 ± 0.004	0.578 ± 0.046	1.10 ± 0.06	0.105 ± 0.005
16	50	200	0.330 ± 0.001	0.643 ± 0.036	1.60 ± 0.12	0.399 ± 0.009
16	200	10	0.326 ± 0.002	0.805 ± 0.421	0.983 ± 0.026	0.089 ± 0.005
16	200	50	0.351 ± 0.001	0.629 ± 0.039	1.70 ± 0.04	0.400 ± 0.008
16	200	200	0.458 ± 0.010	1.25 ± 0.05	2.97 ± 0.12	1.59 ± 0.02
64	10	10	0.379 ± 0.003	0.683 ± 0.027	1.11 ± 0.11	0.065 ± 0.009
64	10	50	0.400 ± 0.003	0.693 ± 0.037	2.13 ± 0.03	0.292 ± 0.014
64	10	200	0.429 ± 0.001	0.897 ± 0.066	5.03 ± 0.25	1.13 ± 0.05
64	50	10	0.439 ± 0.002	0.694 ± 0.057	2.03 ± 0.06	0.299 ± 0.025
64	50	50	0.464 ± 0.003	0.951 ± 0.022	6.05 ± 0.12	1.34 ± 0.04
64	50	200	0.537 ± 0.004	2.23 ± 0.04	19.26 ± 0.16	5.49 ± 0.07
64	200	10	0.488 ± 0.010	1.04 ± 0.06	5.51 ± 0.09	1.11 ± 0.04
64	200	50	0.571 ± 0.004	2.24 ± 0.02	20.03 ± 0.16	5.48 ± 0.09
64	200	200	0.789 ± 0.010	9.72 ± 0.18	70.66 ± 0.00	21.69 ± 0.13

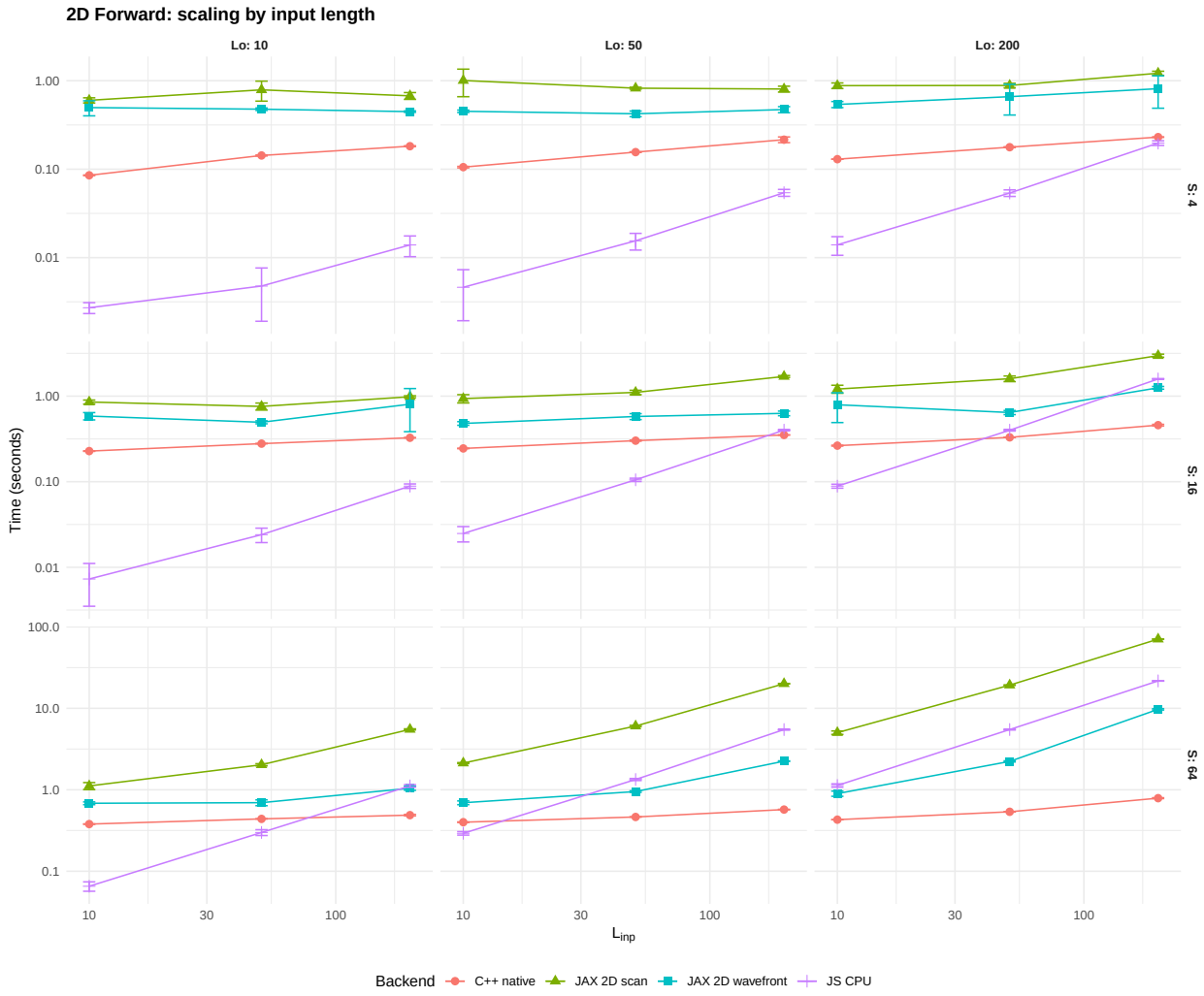

Figure 5: 2D Forward: scaling by input length L_{in}, faceted by states S (rows) and output length L_{out} (columns). Log-log scale.

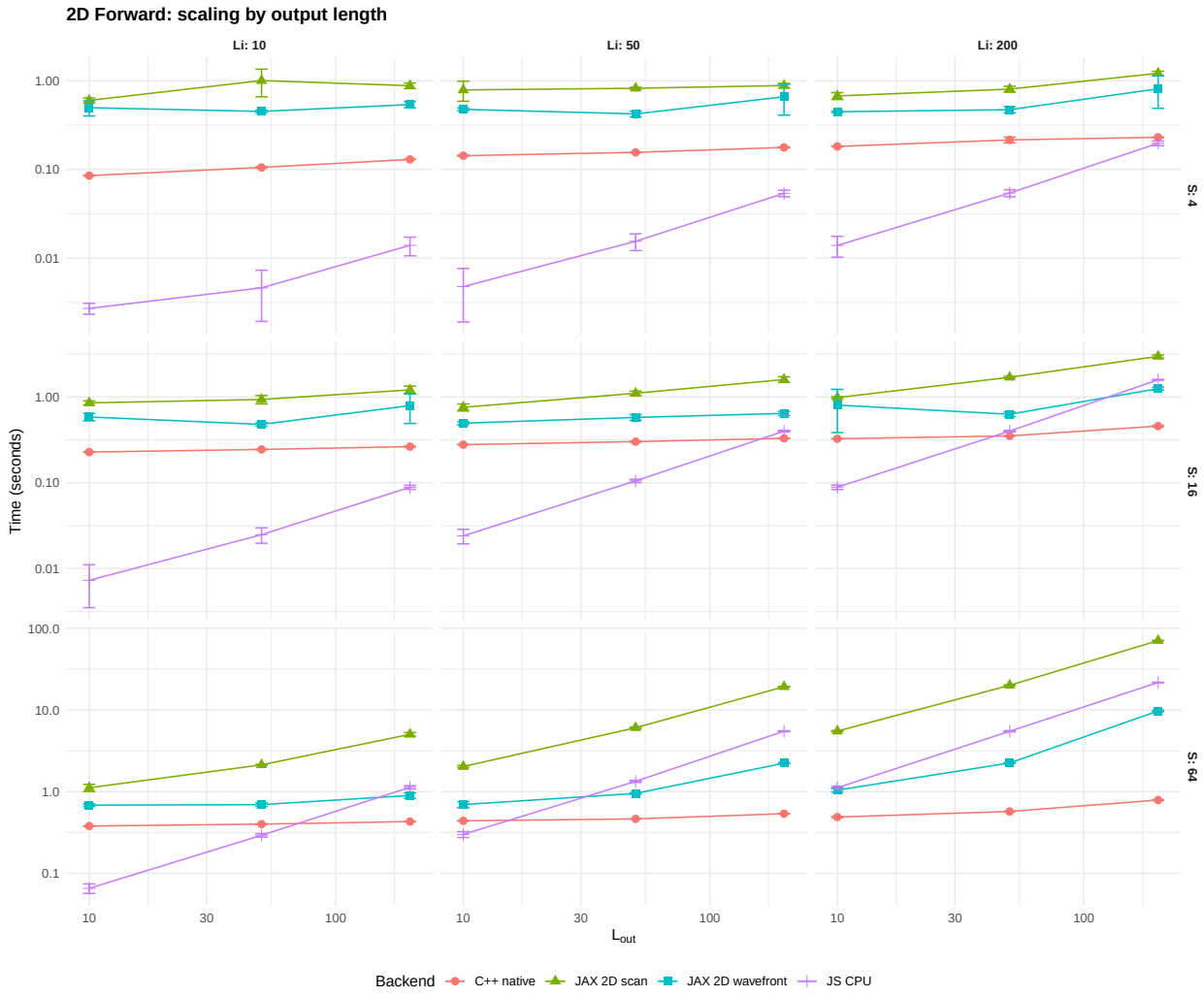

Figure 6: 2D Forward: scaling by output length L_{out}, faceted by states S (rows) and input length L_{in} (columns). Log-log scale.

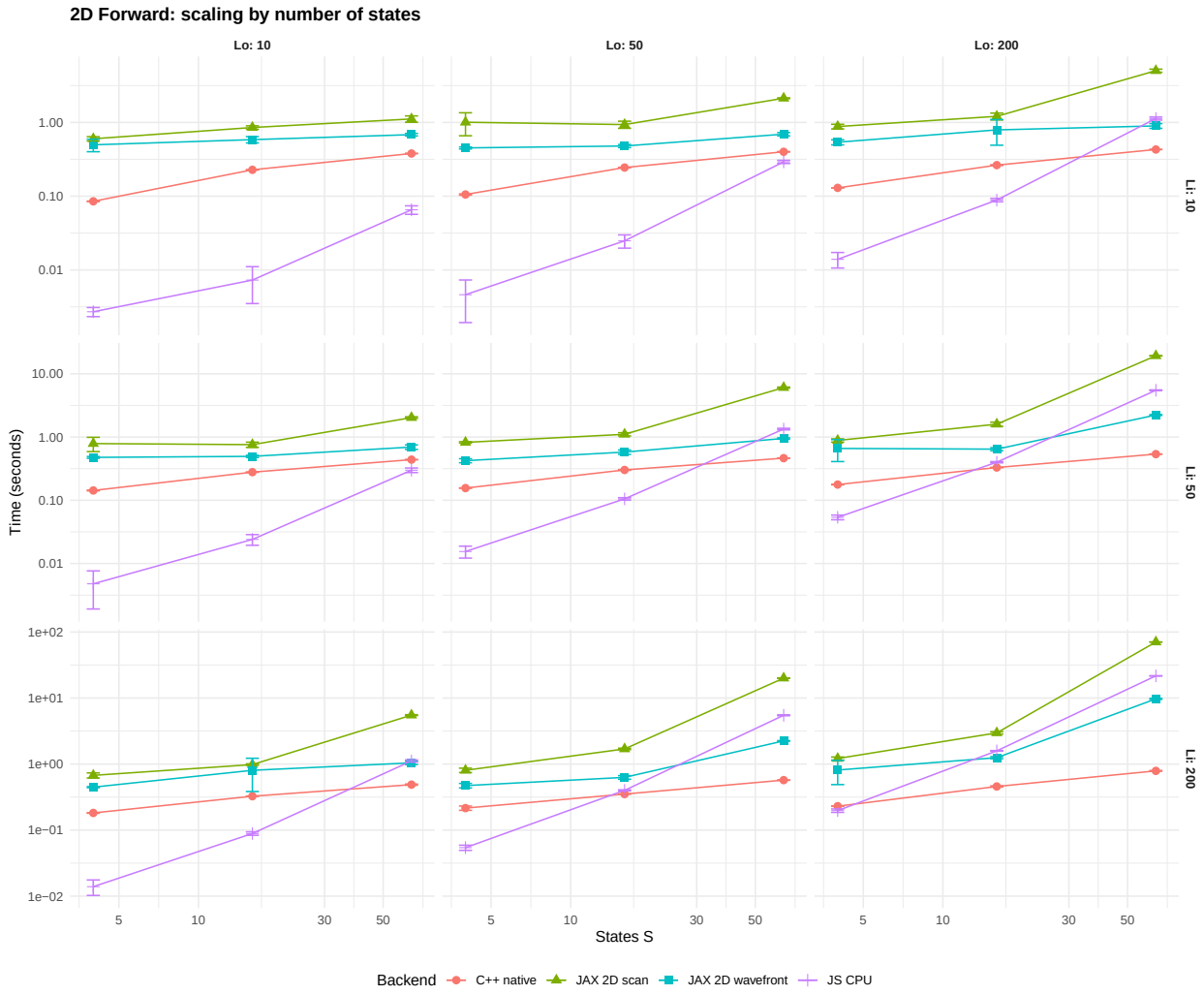

Figure 7: 2D Forward: scaling by states S, faceted by L_{in} (rows) and L_{out} (columns). Log-log scale.

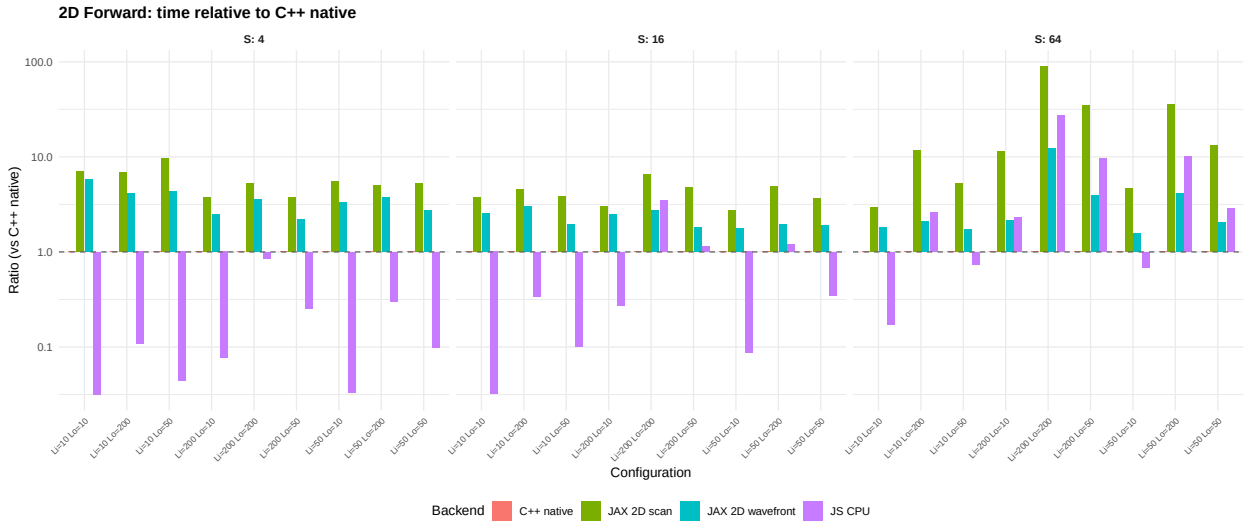

Figure 8: 2D Forward: time relative to C++ native, by configuration.

6 Results: 2D Viterbi

Table 4: 2D Viterbi timings (transducers) (seconds, mean ± std).

S	L_{in}	L_{out}	cpp	jax_2d_optimal	jax_2d_simple	js_cpu
4	10	10	0.110 ± 0.008	0.329 ± 0.032	0.424 ± 0.038	0.0014 ± 0.0003
4	10	50	0.123 ± 0.002	0.320 ± 0.029	0.407 ± 0.022	0.0039 ± 0.0018
4	10	200	0.137 ± 0.001	0.315 ± 0.006	0.477 ± 0.041	0.013 ± 0.002
4	50	10	0.157 ± 0.003	0.305 ± 0.035	0.398 ± 0.026	0.0038 ± 0.0017
4	50	50	0.161 ± 0.002	0.310 ± 0.020	0.397 ± 0.022	0.015 ± 0.002
4	50	200	0.186 ± 0.005	0.299 ± 0.001	0.495 ± 0.028	0.051 ± 0.006
4	200	10	0.198 ± 0.002	0.307 ± 0.030	0.391 ± 0.039	0.014 ± 0.002
4	200	50	0.207 ± 0.001	0.287 ± 0.007	0.451 ± 0.034	0.051 ± 0.006
4	200	200	0.236 ± 0.001	0.396 ± 0.016	0.673 ± 0.003	0.185 ± 0.011
16	10	10	0.241 ± 0.004	0.326 ± 0.020	0.421 ± 0.031	0.0066 ± 0.0027
16	10	50	0.263 ± 0.003	0.340 ± 0.045	0.489 ± 0.036	0.024 ± 0.004
16	10	200	0.293 ± 0.015	0.326 ± 0.029	0.587 ± 0.030	0.100 ± 0.030
16	50	10	0.306 ± 0.022	0.332 ± 0.042	0.427 ± 0.010	0.024 ± 0.004
16	50	50	0.305 ± 0.002	0.413 ± 0.037	0.554 ± 0.055	0.101 ± 0.004
16	50	200	0.333 ± 0.003	0.402 ± 0.025	0.770 ± 0.046	0.376 ± 0.009
16	200	10	0.337 ± 0.004	0.323 ± 0.017	0.503 ± 0.045	0.083 ± 0.005
16	200	50	0.371 ± 0.023	0.397 ± 0.018	0.863 ± 0.028	0.376 ± 0.012
16	200	200	0.430 ± 0.003	0.664 ± 0.011	1.47 ± 0.08	1.44 ± 0.01
64	10	10	0.392 ± 0.001	0.401 ± 0.039	0.585 ± 0.056	0.064 ± 0.006
64	10	50	0.431 ± 0.016	0.731 ± 0.424	1.08 ± 0.04	0.275 ± 0.012
64	10	200	0.453 ± 0.006	0.554 ± 0.014	2.63 ± 0.07	1.08 ± 0.03
64	50	10	0.447 ± 0.003	0.428 ± 0.016	1.08 ± 0.02	0.275 ± 0.012
64	50	50	0.470 ± 0.002	0.540 ± 0.024	3.13 ± 0.07	1.25 ± 0.03
64	50	200	0.539 ± 0.001	0.903 ± 0.004	10.70 ± 0.02	5.03 ± 0.03
64	200	10	0.493 ± 0.002	0.539 ± 0.013	2.84 ± 0.08	1.09 ± 0.01
64	200	50	0.563 ± 0.003	0.875 ± 0.009	10.80 ± 0.06	5.04 ± 0.05
64	200	200	0.717 ± 0.014	3.64 ± 0.01	38.42 ± 0.05	19.74 ± 0.18

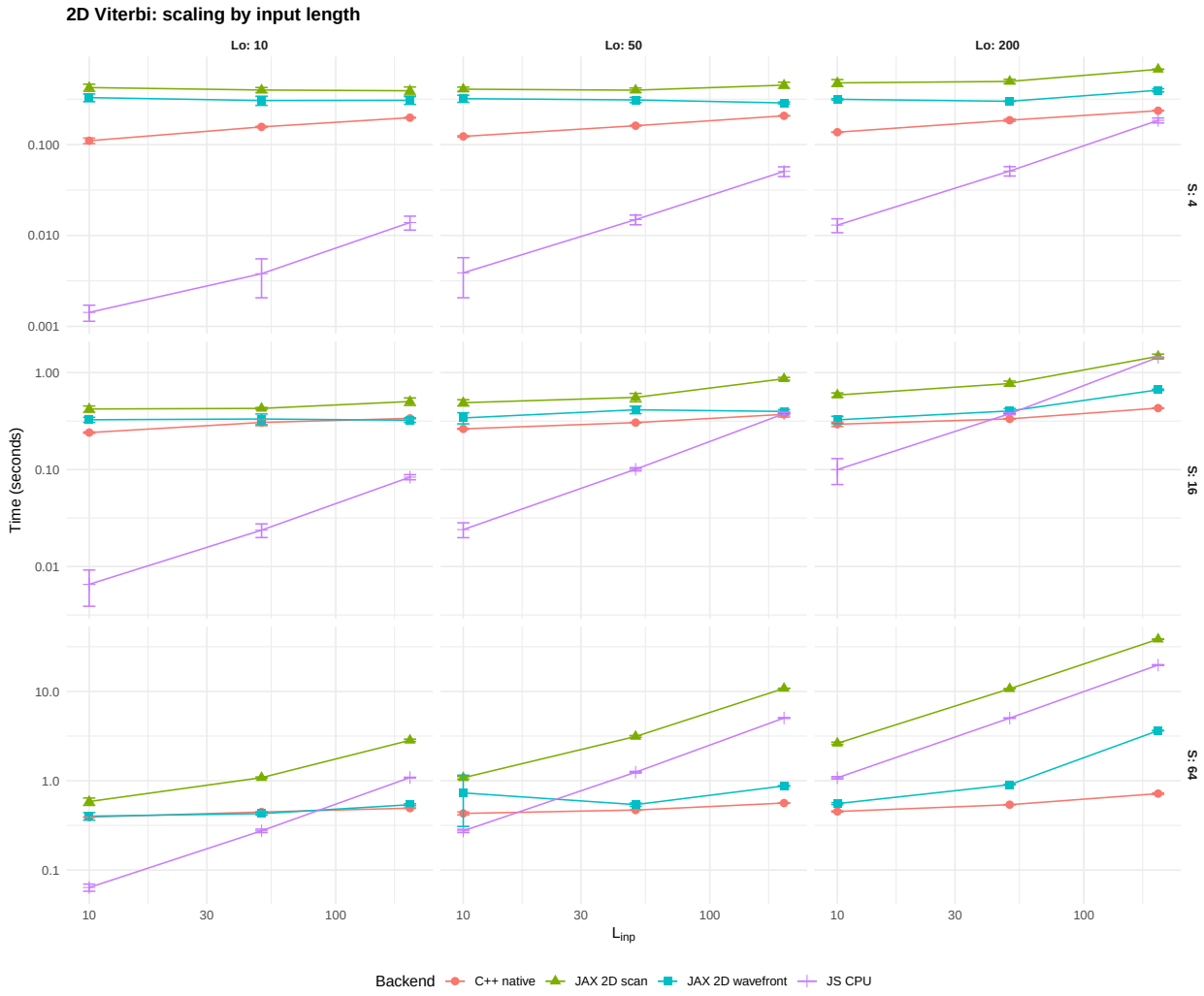

Figure 9: 2D Viterbi: scaling by input length L_{in}, faceted by states S (rows) and output length L_{out} (columns). Log-log scale.

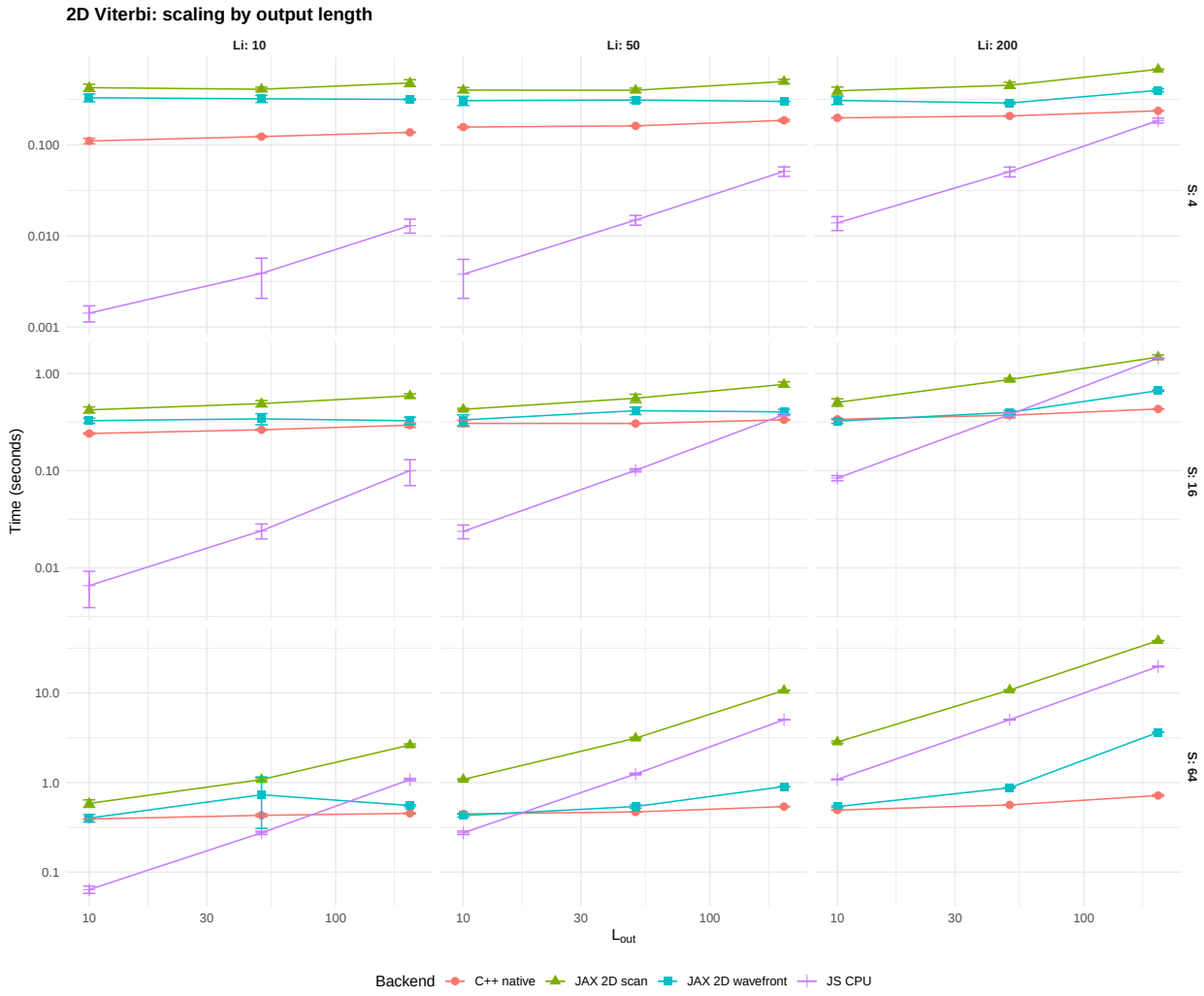

Figure 10: 2D Viterbi: scaling by output length L_{out}, faceted by states S (rows) and input length L_{in} (columns). Log-log scale.

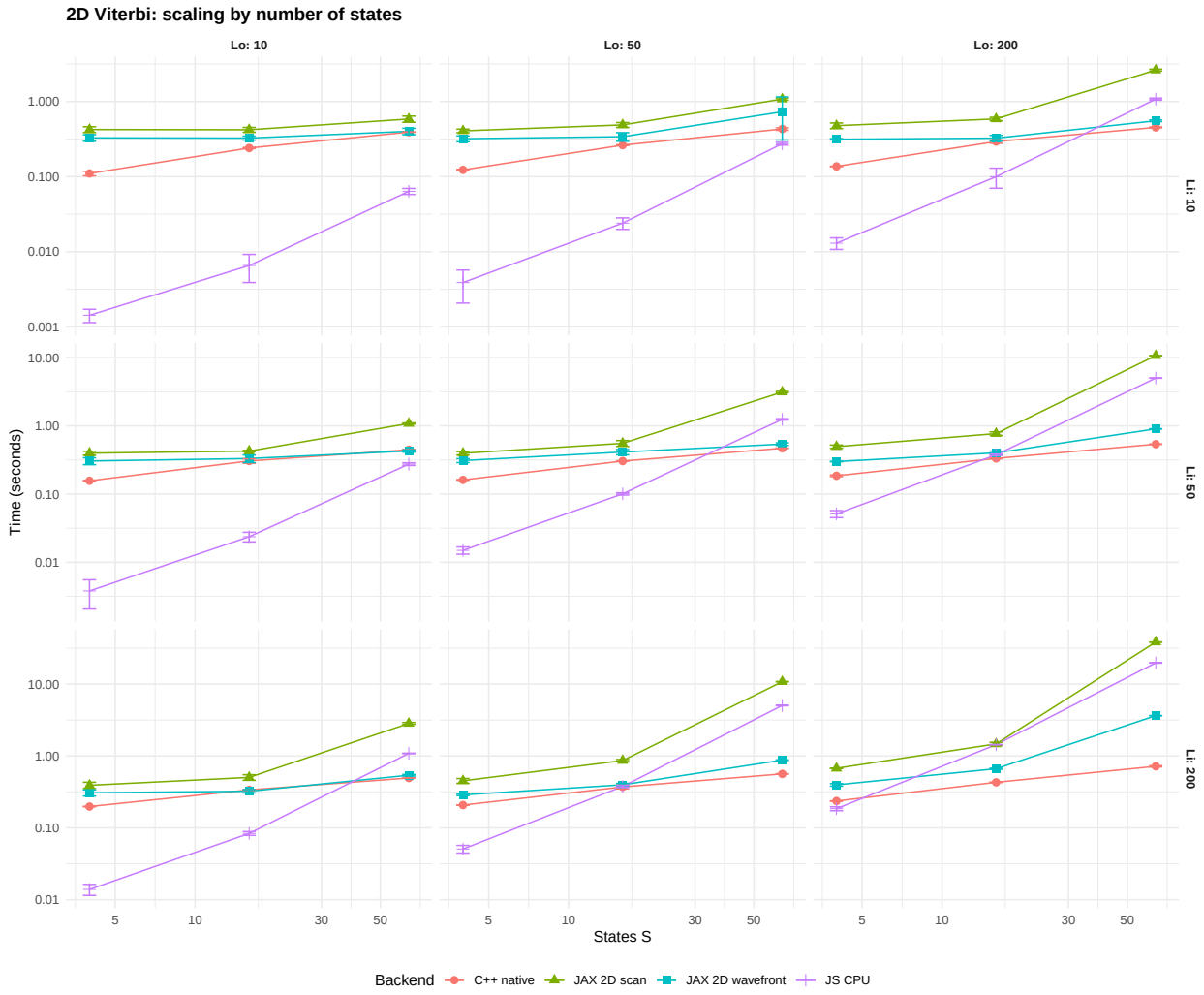

Figure 11: 2D Viterbi: scaling by states S, faceted by L_{in} (rows) and L_{out} (columns). Log-log scale.

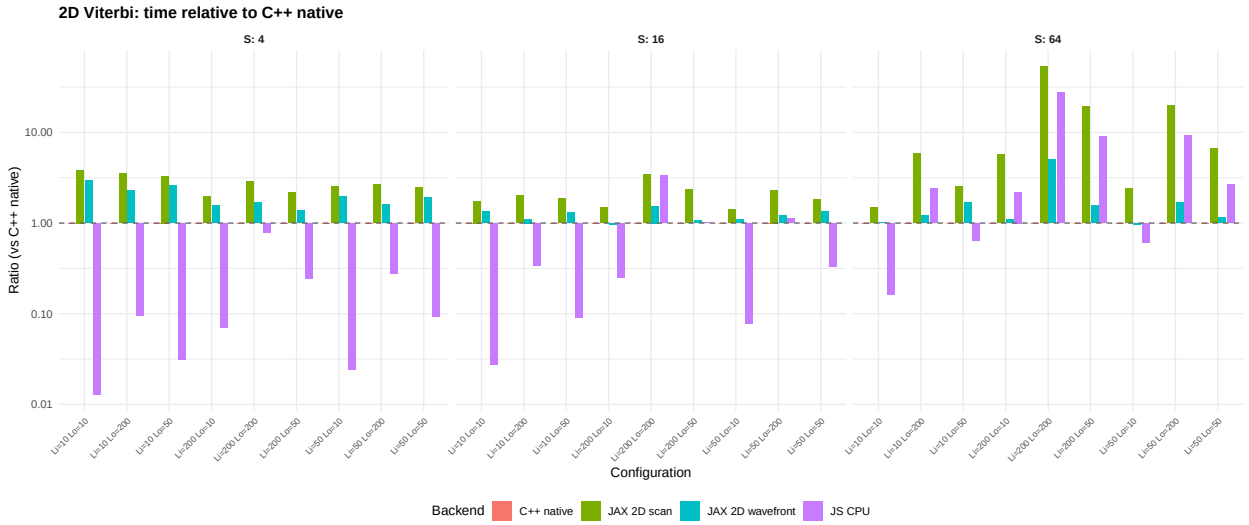

Figure 12: 2D Viterbi: time relative to C++ native, by configuration.